

**МИНОБРНАУКИ РОССИИ**  
**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ**  
**ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**  
**«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)**  
**Кафедра МО ЭВМ**

**ОТЧЕТ**  
**по лабораторной работе №4**  
**по дисциплине «Построение и анализ алгоритмов»**  
**ТЕМА: КНУТ-МОРРИС-ПРАТТ.**

Студент гр. 4343

Селезнев Д. Е.

Преподаватель

Жангиров Т. Р.

Санкт-Петербург

2026

### **Цель работы.**

Изучить алгоритм Кнута – Морриса – Пратта для эффективного поиска подстроки в строке. Реализовать на языке программирования Python алгоритм КМП для нахождения всех вхождений заданного шаблона в текст с использованием префикс – функции. Исследовать применение алгоритма для решения задачи определения циклического сдвига строк. Реализовать алгоритм проверки циклического сдвига двух строк и нахождения индекса соответствующего сдвига. Провести тестирование реализованных алгоритмов на различных входных данных и проверить корректность и эффективность работы программ.

### **Задание.**

#### **Задание №1**

Реализуйте алгоритм КМП и с его помощью для заданных шаблона  $P$  ( $|P| \leq 25000$ ) и текста  $T$  ( $|T| \leq 5000000$ ) найдите все вхождения  $P$  в  $T$ .

Вход:

- Первая строка –  $P$
- Вторая строка –  $T$

Выход:

индексы начал вхождений  $P$  в  $T$ , разделённые запятой; если  $P$  не входит в  $T$ , то вывести -1.

#### **Задание №2**

Заданы две строки  $A$  ( $|A| \leq 5000000$ ) и  $B$  ( $|B| \leq 5000000$ ). Определить, является ли  $A$  циклическим сдвигом  $B$  (это значит, что  $A$  и  $B$  имеют одинаковую длину и  $A$  состоит из суффикса  $B$ , склеенного с префиксом  $B$ ). Например, defabc является циклическим сдвигом abcdef.

Вход:

- Первая строка –  $A$
- Вторая строка –  $B$

Выход:

Если  $A$  является циклическим сдвигом  $B$ , то индекс начала строки  $B$  в  $A$ ; иначе вывести -1. Если возможно несколько сдвигов, вывести первый индекс.

### **Теоретический минимум.**

Алгоритм Кнута – Морриса – Пратта (КМП) предназначен для поиска подстроки в строке за линейное время. Он основан на предварительной обработке шаблона с помощью префикс – функции, которая позволяет эффективно использовать информацию о совпадающих префиксах и суффиксах строки. Благодаря этому алгоритм избегает повторных сравнений уже проверенных символов и обеспечивает линейную сложность обработки.

Основная идея КМП заключается в том, что при несовпадении символов поиск не начинается заново, а продолжается с позиции, определённой значением префикс-функции. Это позволяет существенно ускорить процесс поиска и делает алгоритм применимым для обработки очень длинных строк.

В данной лабораторной работе алгоритм КМП используется как основа для решения двух задач:

- поиска всех вхождений шаблона в тексте;
- определения циклического сдвига строки.

### **Префикс-функция**

Префикс-функция  $\pi[i]$  для строки  $P$  определяется как длина наибольшего собственного префикса подстроки  $P[0..i]$ , который одновременно является её суффиксом.

Иными словами, префикс-функция показывает, насколько можно “сдвинуть” строку при несовпадении, сохранив уже найденное частичное совпадение.

Основная идея вычисления префикс-функции:

- для каждого символа строки поддерживается значение текущего совпадения  $j$ ;
- при совпадении символов значение увеличивается;

- при несовпадении происходит переход по ранее вычисленному значению  $\pi[j - 1]$ , что позволяет использовать уже найденные совпадения без повторного анализа.

Таким образом формируется массив  $\pi$ , который далее используется в алгоритме поиска.

### **Поиск всех вхождений подстроки**

После построения префикс – функции выполняется основной этап алгоритма – поиск шаблона в тексте.

Основная идея работы:

- выполняется последовательный проход по строке текста  $T$ ;
- символы текста сравниваются с символами шаблона  $P$ ;
- при совпадении увеличивается текущая длина совпадения;
- при несовпадении происходит переход по префикс – функции, что позволяет продолжить поиск без возврата к началу шаблона;
- при достижении длины шаблона фиксируется позиция вхождения.

Результатом работы алгоритма является список всех индексов начала вхождений шаблона в текст. Если совпадений не найдено, возвращается значение -1.

### **Циклический сдвиг строки**

Циклический сдвиг строки – это преобразование, при котором строка представляется как конкатенация её суффикса и префикса. Например, строка  $abcdef$  может быть преобразована в  $defabc$ .

Во второй задаче рассматривается определение того, является ли строка  $A$  циклическим сдвигом строки  $B$ .

Основная идея решения заключается в использовании алгоритма КМП с модификацией:

- строка  $B$  используется как шаблон;
- строка  $A$  рассматривается как циклическая последовательность, то есть символы берутся по индексу  $i \bmod n$ ;

- выполняется поиск полного совпадения шаблона  $B$  в этой циклической структуре;
- если совпадение найдено, возвращается индекс начала сдвига, иначе результатом является  $-1$ .

Таким образом, задача сводится к поиску подстроки в циклически расширенном тексте, что позволяет использовать линейный алгоритм КМП без увеличения сложности.

### **Временная сложность**

- построение префикс-функции:  $O(m)$
- поиск подстроки:  $O(n)$
- алгоритм КМП в целом:  $O(m + n)$

Для циклического сдвига (при  $m = n$ ):  $O(n)$

### **Пространственная сложность**

- хранение префикс-функции:  $O(m)$
- дополнительные переменные:  $O(1)$
- итоговая сложность:  $O(m)$

Для циклического сдвига:  $O(n)$

### **Выполнение работы.**

#### **1. Поиск всех вхождений шаблона в тексте (КМП)**

Программа реализует алгоритм Кнута – Морриса – Пратта для поиска всех вхождений шаблона  $P$  в тексте  $T$ . Алгоритм использует префикс – функцию для эффективного поиска и гарантирует линейное время работы.

*compute\_prefix\_function(pattern)* – функция вычисления префикс-функции для шаблона. В начале определяется длина шаблона  $m$  и создаётся массив  $pi$  размера  $m$ , заполненный нулями. Далее выполняется цикл от  $i = 1$  до  $m - 1$ . На каждой итерации берётся значение  $j = pi[i - 1]$ . Цикл *while* выполняет откаты: пока  $j > 0$  и  $pattern[i] \neq pattern[j]$ , выполняется  $j = pi[j - 1]$ . Если после откатов  $pattern[i] = pattern[j]$ , то  $j$  увеличивается на

1. Значение  $pi[i]$  устанавливается равным  $j$ . Итоговый массив  $pi$  возвращается как результат.

$kmp\_search(pattern, text)$  – функция поиска всех вхождений шаблона в тексте. Сначала проверяются граничные случаи: если  $pattern$  или  $text$  пусты, возвращается пустой список. Далее вычисляется префикс – функция для шаблона. Инициализируется пустой список  $matches$  для хранения индексов вхождений и переменная  $j = 0$  для отслеживания количества совпавших символов. Цикл проходит по всем символам текста. На каждой итерации выполняются откаты: пока  $j > 0$  и  $text[i] \neq pattern[j]$ , выполняется  $j = pi[j - 1]$ . Если  $text[i] = pattern[j]$ , то  $j$  увеличивается на 1. Если  $j = m$  (весь шаблон совпал), в список  $matches$  добавляется индекс начала вхождения  $i - m + 1$ , и выполняется откат  $j = pi[j - 1]$  для продолжения поиска. После завершения цикла возвращается список всех найденных вхождений.

### Пайплайн

В блоке `if __name__ == "__main__":` программа считывает две строки: шаблон  $P$  и текст  $T$ . Предобработка входных данных включает удаление пробельных символов с помощью `strip()`. Далее вызывается функция `kmp_search(P, T)`. Постобработка проверяет наличие вхождений: если список  $matches$  пуст, формируется строка "-1", иначе индексы объединяются через запятую с помощью `','.join(map(str, matches))`. Вывод осуществляется с помощью `print` – на экран выводятся индексы начал всех вхождений или -1.

### 2. Проверка циклического сдвига

Программа реализует проверку, является ли строка  $A$  циклическим сдвигом строки  $B$ , используя алгоритм КМП с виртуальным доступом к строке  $A + A$ .

$compute\_prefix\_function(pattern)$  – функция вычисления префикс-функции, аналогичная первой задаче

$kmp\_search\_cyclic(A, B)$  – функция поиска  $B$  в виртуальной строке  $A + A$ . Определяются длины строк  $m = len(B)$  и  $n = len(A)$ . Вычисляется префикс – функция для  $B$ . Инициализируется  $j = 0$ . Цикл проходит от  $i = 0$

до  $2n - 1$ , имитируя проход по строке  $A + A$ . На каждой итерации получается символ  $current\_char = A[i \% n]$  через модульную арифметику (виртуальный доступ). Выполняются откаты: пока  $j > 0$  и  $current\_char \neq B[j]$ , выполняется  $j = pi[j - 1]$ . Если  $current\_char = B[j]$ , то  $j$  увеличивается на 1. Если  $j = m$  (весь шаблон  $B$  совпал), возвращается индекс начала вхождения  $i - m + 1$ . Если цикл завершился без нахождения совпадения, возвращается -1.

$find\_cyclic\_shift(A, B)$  – главная функция проверки циклического сдвига. Сначала проверяется равенство длин: если  $len(A) \neq len(B)$ , возвращается -1. Если  $len(A) = 0$  (обе строки пустые), возвращается 0. Далее вызывается  $kmp\_search\_cyclic(A, B)$  и возвращается полученный индекс.

### **Пайплайн**

В блоке `if __name__ == "__main__":` программа считывает две строки:  $A$  и  $B$ . Предобработка входных данных включает удаление пробельных символов с помощью `strip()`. Далее вызывается функция  $find\_cyclic\_shift(A, B)$ . Постобработка не требуется, так как результат уже содержит нужное значение индекса или -1. Вывод осуществляется с помощью `print` – на экран выводится индекс начала строки  $B$  в  $A$  или -1, если  $A$  не является циклическим сдвигом  $B$ .

### **Тестирование.**

#### **Поиск всех вхождений шаблона в тексте (КМП)**

Для проверки корректности реализации алгоритма КМП был разработан набор модульных тестов с использованием фреймворка `pytest`. Тестирование разделено на две группы: проверка префикс-функции и проверка поиска вхождений.

Тестирование префикс – функции включает проверку базовых случаев: простой шаблон без повторений, где все значения префикс-функции равны нулю; шаблон с повторяющимися символами, демонстрирующий корректное вычисление совпадающих префиксов и суффиксов; а также случай с

полностью одинаковыми символами, где значения префикс – функции последовательно увеличиваются.

Тестирование поиска вхождений включает проверку базового случая из условия задачи с перекрывающимися вхождениями шаблона в тексте. Проверяются множественные вхождения без перекрытий, а также случай отсутствия вхождений. Отдельное внимание уделяется перекрывающимся вхождениям, где шаблон встречается с наложением. Также проверяются граничные случаи: шаблон длиннее текста, пустой шаблон и пустой текст, при которых алгоритм должен корректно возвращать пустой список.

### **Проверка циклического сдвига**

Для проверки корректности реализации алгоритма проверки циклического сдвига был разработан набор модульных тестов с использованием *pytest*. Тестирование включает проверку базового случая из условия задачи, где строка является циклическим сдвигом другой строки на определённое количество позиций. Проверяется случай идентичных строк, при котором индекс сдвига должен быть равен нулю. Дополнительно тестируются различные варианты циклических сдвигов с разными индексами, подтверждающие корректность работы алгоритма на различных входных данных.

Тестирование также включает проверку негативных случаев: строки, не являющиеся циклическими сдвигами друг друга, должны возвращать -1; строки разной длины также должны возвращать -1, так как циклический сдвиг возможен только для строк одинаковой длины. Отдельное внимание уделяется граничному случаю с пустыми строками, при котором алгоритм должен корректно возвращать индекс 0.

Программный код реализации тестов см. в приложении А.

Результаты тестирования см. в приложении Б.

### **Вывод.**

В ходе выполнения лабораторной работы были реализованы алгоритм Кнута – Морриса – Пратта для поиска всех вхождений шаблона в тексте, а



также алгоритм проверки циклического сдвига строк с использованием КМП и виртуального доступа к строке.

Алгоритм КМП позволяет эффективно находить все вхождения шаблона в тексте за линейное время с использованием префикс – функции. Реализация основана на построении префикс – функции для шаблона, которая определяет длину наибольшего собственного префикса, совпадающего с суффиксом для каждой позиции. Это позволяет избежать повторных сравнений уже проверенных символов текста при несовпадении, обеспечивая временную сложность  $O(m + n)$ , где  $m$  – длина шаблона,  $n$  – длина текста.

Также реализован алгоритм проверки циклического сдвига, который определяет, является ли одна строка циклическим сдвигом другой. Реализация основана на ключевой идее: строка  $A$  является циклическим сдвигом строки  $B$  тогда и только тогда, когда  $B$  встречается в виртуальной строке  $A + A$ . Для оптимизации памяти используется виртуальный доступ через модульную арифметику вместо явного создания строки  $A + A$ , что позволяет снизить пространственную сложность с  $O(2n)$  до  $O(1)$  для текста.

Корректность реализованных алгоритмов подтверждена с помощью набора модульных тестов, проверяющих граничные случаи (пустые строки, строки разной длины), стандартные ситуации (множественные и перекрывающиеся вхождения, различные циклические сдвиги) и негативные случаи (отсутствие вхождений, строки, не являющиеся циклическими сдвигами). Все тесты успешно пройдены, что подтверждает корректность реализации алгоритмов.

Таким образом, в ходе работы были изучены и реализованы основные методы решения задач поиска подстроки и проверки циклического сдвига на основе алгоритма Кнута – Морриса – Пратта, исследованы их особенности и оптимизации, а также подтверждена корректность их работы на тестовых данных.

## ПРИЛОЖЕНИЕ А

### ИСХОДНЫЙ КОД ПРОГРАММЫ

#### **kmp\_search.py**

```
def compute_prefix_function(pattern):
    m = len(pattern)
    pi = [0] * m

    for i in range(1, m):
        j = pi[i - 1]

        while j > 0 and pattern[i] != pattern[j]:
            j = pi[j - 1]

        if pattern[i] == pattern[j]:
            j += 1

        pi[i] = j

    return pi

def kmp_search(pattern, text):
    if not pattern or not text:
        return []

    m = len(pattern)
    n = len(text)

    pi = compute_prefix_function(pattern)

    matches = []
    j = 0

    for i in range(n):
        while j > 0 and text[i] != pattern[j]:
            j = pi[j - 1]

        if text[i] == pattern[j]:
            j += 1

        if j == m:
            matches.append(i - m + 1)
            j = pi[j - 1]

    return matches

if __name__ == "__main__":
    pattern = input().strip()
    text = input().strip()

    matches = kmp_search(pattern, text)

    if matches:
        print(', '.join(map(str, matches)))
    else:
        print(-1)
```

## **cyclic\_shift.py**

```
def compute_prefix_function(pattern):
    m = len(pattern)
    pi = [0] * m

    for i in range(1, m):
        j = pi[i - 1]

        while j > 0 and pattern[i] != pattern[j]:
            j = pi[j - 1]

        if pattern[i] == pattern[j]:
            j += 1

        pi[i] = j

    return pi

def kmp_search_cyclic(A, B):
    m = len(B)
    n = len(A)

    pi = compute_prefix_function(B)
    j = 0

    for i in range(2 * n):
        current_char = A[i % n]

        while j > 0 and current_char != B[j]:
            j = pi[j - 1]

        if current_char == B[j]:
            j += 1

        if j == m:
            return i - m + 1

    return -1

def find_cyclic_shift(A, B):
    if len(A) != len(B):
        return -1

    if len(A) == 0:
        return 0

    index = kmp_search_cyclic(A, B)

    return index

if __name__ == "__main__":
    A = input().strip()
    B = input().strip()

    result = find_cyclic_shift(A, B)
    print(result)
```

## **test\_kmp\_search.py**

```
import pytest
from kmp_search import compute_prefix_function, kmp_search

class TestPrefixFunction:
    def test_simple_pattern(self):
        assert compute_prefix_function("abc") == [0, 0, 0]

    def test_pattern_with_repetition(self):
        assert compute_prefix_function("abab") == [0, 0, 1, 2]

    def test_pattern_all_same(self):
        assert compute_prefix_function("aaaa") == [0, 1, 2, 3]

class TestKMPSearch:
    def test_basic_case(self):
        assert kmp_search("ab", "abab") == [0, 2]

    def test_multiple_occurrences(self):
        assert kmp_search("abc", "abcabcabc") == [0, 3, 6]

    def test_no_occurrences(self):
        assert kmp_search("xyz", "abcdef") == []

    def test_overlapping_occurrences(self):
        assert kmp_search("aa", "aaaa") == [0, 1, 2]

    def test_pattern_longer_than_text(self):
        assert kmp_search("abcdef", "abc") == []

    def test_empty_pattern(self):
        assert kmp_search("", "abc") == []

    def test_empty_text(self):
        assert kmp_search("abc", "") == []
```

## **test\_cyclic\_shift.py**

```
import pytest
from cyclic_shift import compute_prefix_function, kmp_search_cyclic,
find_cyclic_shift

class TestCyclicShift:
    def test_basic_case(self):
        assert find_cyclic_shift("defabc", "abcdef") == 3

    def test_identical_strings(self):
        assert find_cyclic_shift("abcdef", "abcdef") == 0

    def test_another_shift(self):
        assert find_cyclic_shift("bcdefa", "abcdef") == 5

    def test_not_cyclic_shift(self):
        assert find_cyclic_shift("abcxyz", "abcdef") == -1

    def test_different_lengths(self):
        assert find_cyclic_shift("abc", "abcdef") == -1
```

```
def test_empty_strings(self):  
    assert find_cyclic_shift("", "") == 0
```

## ПРИЛОЖЕНИЕ Б

### ТЕСТИРОВАНИЕ

Таблица 1 - Результаты тестирования алгоритма КМП (поиск вхождений)

| № п/п | Входные данные                    | Выходные данные | Комментарии                                        |
|-------|-----------------------------------|-----------------|----------------------------------------------------|
| 1.    | pattern="abc"                     | []              | Простой шаблон без повторений                      |
| 2.    | pattern="abab"                    | [0, 0, 1, 2]    | Шаблон с повторяющимися символами                  |
| 3.    | pattern="aaaa"                    | [0, 1, 2, 3]    | Все символы одинаковые                             |
| 4.    | pattern="ab"<br>text="abab"       | [0, 2]          | Корректная работа базового теста из условия задачи |
| 5.    | pattern="abc"<br>text="abcabcabc" | [0, 3, 6]       | Множественные вхождения без перекрытий             |
| 6.    | pattern="xyz"<br>text="abcdef"    | []              | Корректная работа при отсутствии вхождений         |
| 7.    | pattern="aa"<br>text="aaaa"       | [0, 1, 2]       | Перекрывающиеся вхождения                          |
| 8.    | pattern="abcdef"<br>text="abc"    | []              | Шаблон длиннее текста                              |
| 9.    | pattern=""<br>text="abc"          | []              | Корректная работа при пустом шаблоне               |
| 10.   | pattern="abc"<br>text=""          | []              | Корректная работа при пустом тексте                |

Таблица 2 - Результаты тестирования алгоритма проверки циклического сдвига

| № п/п | Входные данные           | Выходные данные | Комментарии                                        |
|-------|--------------------------|-----------------|----------------------------------------------------|
| 1.    | A="defabc"<br>B="abcdef" | 3               | Базовый тест из условия задачи                     |
| 2.    | A="abcdef"<br>B="abcdef" | 0               | Корректная работа при идентичных строках (сдвиг 0) |
| 3.    | A="bcdefa"<br>B="abcdef" | 5               | Другой вариант циклического сдвига                 |
| 4.    | A="abcxyz"<br>B="abcdef" | -1              | Строки не являются циклическими сдвигами           |
| 5.    | A="abc"<br>B="abcdef"    | -1              | Корректная работа при строках разной длины         |
| 6.    | A=""<br>B=""             | 0               | Пустые строки                                      |